

ZK-ACE: Identity-Centric Zero-Knowledge Authorization for Post-Quantum Blockchain Systems

Jian Sheng Wang
Yeah LLC

February 26, 2026

Revised March 13, 2026: updated constraint counts from reference implementation; added benchmarks (Section 12).

Abstract

Post-quantum signature schemes introduce kilobyte-scale authorization artifacts when applied directly to blockchain transaction validation. A widely considered mitigation is to verify post-quantum signatures inside zero-knowledge circuits and publish only succinct proofs on-chain. However, this approach preserves the signature-centric authorization model, merely relocating the verification cost, and embeds expensive high-dimensional lattice arithmetic into prover circuits.

We present ZK-ACE (**Z**ero-**K**nowledge **A**uthorization for **C**ryptographic **E**ntities), an authorization layer that replaces transaction-carried signature objects entirely with identity-bound zero-knowledge authorization statements. Rather than proving the correctness of a specific post-quantum signature, the prover demonstrates in zero knowledge that a transaction is authorized by an identity consistent with an on-chain commitment and bound replay state. The construction assumes a *deterministic identity derivation primitive* (DIDP) as a black box and uses a compact identity commitment as the primary on-chain identity anchor, supplemented by per-transaction replay-prevention state.

We formalize ZK-ACE with explicit game-based security definitions for authorization soundness, replay resistance, substitution resistance, and cross-domain separation. We present a complete circuit constraint specification, define two replay-prevention models (nonce-registry and nullifier-set), and provide reduction-based security proofs under standard assumptions (knowledge soundness, collision resistance, and DIDP identity-root recovery hardness). A structural, protocol-level data accounting demonstrates an order-of-magnitude reduction in consensus-visible authorization data relative to direct post-quantum signature deployment. The design supports batch aggregation and recursive proof composition, and is compatible with account-abstraction and rollup-based deployment architectures.

Keywords. Post-Quantum Cryptography, Deterministic Identity, Zero-Knowledge Proofs, Identity Commitment, Blockchain Authorization, Lattice-Based Signatures, Account Abstraction.

1 Introduction

1.1 Post-Quantum Signatures and Blockchain Scalability

The standardization of post-quantum cryptography (PQC) by NIST represents a necessary step toward long-term security in the presence of quantum adversaries. Lattice-based signature schemes such as ML-DSA (formerly Dilithium) [2] and hash-based schemes such as SLH-DSA

(formerly SPHINCS⁺) [3] have emerged as leading candidates to replace classical public-key signatures.

While these schemes are well-suited for conventional communication settings, their direct application to blockchain systems exposes a fundamental tension between post-quantum security and scalability. A distinguishing characteristic of lattice-based signatures is their *large signature size*, typically on the order of several kilobytes. In blockchain environments, where transaction validity must be verified by all consensus participants and stored as part of the permanent ledger, such signature sizes translate directly into increased on-chain data growth. ML-DSA signatures at NIST security level 2 are approximately 2.4 KB, compared to 64 bytes for Ed25519 or 71 bytes for secp256k1 ECDSA—a roughly 30–40× increase per transaction.

This pressure is further amplified in modern scaling architectures. In rollup-based systems, transaction data is posted as calldata or blob data to the base layer, where bandwidth and availability are explicitly constrained and economically priced [17]. Large post-quantum signatures therefore reduce effective transaction throughput and increase marginal costs per transaction. Crucially, this scalability pressure arises from the *structure of the authorization artifacts themselves*, rather than from inefficiencies in any particular implementation.

1.2 Limitations of ZK-Compressed Signature Verification

A natural response to the data-expansion problem is to move post-quantum signature verification off-chain and replace it with zero-knowledge proofs. Under this approach, a prover generates a zero-knowledge proof attesting to the validity of a post-quantum signature, and the blockchain verifies only the succinct proof [18].

However, this strategy encounters a critical limitation: verifying lattice-based signatures requires arithmetic over high-dimensional algebraic structures, which translates into a large number of constraints when expressed inside zero-knowledge circuits. Even when technically feasible, embedding lattice verification logic into ZK circuits significantly increases the proving cost and often shifts the scalability burden from on-chain data to off-chain computation and verification [9, 7].

As a result, ZK-verification of post-quantum signatures risks becoming a performance bottleneck in its own right. Rather than addressing the root cause of the scalability issue, this approach preserves the signature-centric model of authorization and merely relocates its computational cost. This observation motivates a more fundamental question: *should post-quantum signature verification be treated as a mandatory component of on-chain authorization at all?*

1.3 Key Observation: Authorization versus Signature Objects

At the consensus layer, blockchains do not inherently require verification of a specific cryptographic signature object. What consensus requires is assurance that a given transaction was *authorized* by the correct entity under the system’s rules.

In traditional designs, cryptographic signatures serve as the mechanism by which authorization is expressed and verified. However, signatures are ultimately an *implementation artifact* rather than the authorization semantics themselves. Treating the signature object as the primary unit of verification implicitly couples consensus security to the concrete properties of a particular signature scheme—including its key sizes, signature sizes, and verification costs.

This work adopts a different viewpoint: authorization should be modeled as a *semantic property*—that a specific identity has approved a specific transaction—independent of the concrete cryptographic artifact traditionally used to express that approval. From this perspective, the role of zero-knowledge proofs is not to verify the correctness of a post-quantum signature object, but to prove that the underlying authorization condition holds.

1.4 Our Setting and Scope

We consider a setting in which a *deterministic identity derivation primitive* (DIDP) is assumed to exist as a cryptographic building block.¹ A DIDP provides a stable, deterministic mapping from a 256-bit Root Entropy Value (REV) to context-specific cryptographic keys, potentially across multiple cryptographic schemes, including post-quantum constructions. Its security properties—including determinism, context isolation, and resistance to identity-root recovery—are taken as given. In this respect, a DIDP is treated analogously to other commonly assumed primitives such as verifiable random functions (VRFs), key derivation functions (KDFs), or cryptographic commitment schemes, which are routinely used as building blocks without re-proving their security in every application.

The present work does not propose, analyze, or modify any specific DIDP construction. Our contribution begins at the layer above the DIDP: given a deterministic identity derivation primitive, how should transaction authorization be represented, proven, and verified in a post-quantum-ready blockchain system?

1.5 Contributions

This paper makes the following contributions:

1. We introduce ZK-ACE, a zero-knowledge authorization layer built on top of deterministic identity derivation, in which transaction authorization is proven via zero-knowledge proofs rather than explicit post-quantum signature objects.
2. We propose an on-chain data model consisting of a compact identity commitment and per-transaction zero-knowledge authorization proofs, replacing kilobyte-scale signature artifacts with constant-size verification material.
3. We formally specify the zero-knowledge circuit statement underlying ZK-ACE, including its public inputs, private witnesses, and five core constraints.
4. We define game-based security notions—authorization soundness, replay resistance, substitution resistance, and cross-domain separation—and provide reduction-based security arguments under standard assumptions (knowledge soundness, collision resistance, DIDP identity-root recovery hardness).
5. We describe two replay-protection mechanisms (nonce-registry and nullifier-set) with a formal analysis of their respective trade-offs.
6. We present aggregation and recursion strategies that enable batch authorization and scalable verification for high-throughput systems.
7. We provide a structural, protocol-level accounting of on-chain authorization data requirements, demonstrating an order-of-magnitude reduction relative to signature-based post-quantum authorization.
8. We report a reference implementation and single-threaded CPU benchmarks validating the constraint counts and demonstrating sub-millisecond verification and sub-100 ms proving on consumer hardware.

1.6 Design Goals

The authorization model should satisfy the following properties:

- **Security.** Authorization must be cryptographically sound and resistant to classical and quantum adversaries within the assumed threat model.
- **Verifiability.** Transaction validity must be efficiently verifiable by consensus participants using only public information.

¹ACE-GF [1] is one concrete instantiation that satisfies the DIDP interface and security properties defined in Section 3.2. Any framework providing the same interface—a deterministic mapping from a high-entropy root to context-specific keys with the properties of determinism, context isolation, and identity-root recovery hardness—may be used in its place.

- **Scalability.** On-chain authorization data and verification cost should remain constant with respect to the size of post-quantum signature objects.
- **Migratability.** Existing identities should be able to transition to the new authorization model without disrupting established recovery or identity assumptions. Identity commitments should be proof-system-agnostic so that verifier and prover upgrades do not require identity rotation.
- **Aggregatability.** The authorization mechanism should admit batching and recursive composition to support high-throughput systems.

1.7 Paper Organization

Section 2 surveys related work. Section 3 introduces notation, the assumed DIDP interface, and the threat model. Section 4 describes the system model and transaction flow. Section 5 defines the identity commitment and binding rules. Section 6 clarifies the authorization semantics. Section 7 specifies the ZK circuit. Section 8 presents formal security definitions and proofs. Section 9 discusses on-chain verification and deployment. Section 10 provides the structural data accounting. Section 11 describes scalability extensions. Section 12 reports a reference implementation and benchmarks. Section 13 addresses limitations, and Section 14 concludes.

2 Related Work

Post-quantum signatures on blockchains. Several efforts have investigated the direct integration of lattice-based or hash-based signatures into blockchain transaction validation [2, 3, 4]. The Ethereum community has explored PQC migration paths within its long-term roadmap. The primary obstacle recognized in these efforts is the significant per-transaction data overhead. Our work addresses this obstacle at the protocol architecture level rather than attempting to optimize signature parameters.

ZK-compressed signature verification. An alternative line of research embeds classical or post-quantum signature verification inside zero-knowledge circuits. Notable examples include Circom-based ECDSA verification [18] and explorations of lattice signature verification within SNARK circuits. While technically feasible for classical signatures whose verification is algebraically lightweight, extending this approach to lattice-based schemes significantly inflates the constraint count and prover cost. ZK-ACE avoids this entirely by not placing any signature verification logic inside the circuit.

Transparent and hash-based proof systems. STARKs [9] and FRI-based interactive oracle proofs provide transparent, plausibly post-quantum-secure proof generation without a trusted setup. Recent work by Chiesa, Fenzi, and Weissenberg [25] constructs zero-knowledge IOPPs for constrained interleaved codes with near-optimal $(1 + o(1))$ soundness-to-proof-length overhead, further advancing the efficiency of hash-based proof systems. These developments improve the *proof system layer* and are complementary to ZK-ACE, which operates at the *authorization model layer*. Even with an optimal transparent proof system, embedding lattice-based signature verification into a ZK circuit still requires expressing high-dimensional polynomial arithmetic (NTTs, non-native modular reductions) as arithmetic constraints—yielding circuits on the order of millions of constraints. ZK-ACE sidesteps this bottleneck entirely: its circuit requires only a small number of ZK-friendly hash evaluations ($\approx 4,000$ R1CS constraints in the reference Poseidon parameterization; see Section 7.3), and is compatible with any backend proof system, including hash-based and transparent constructions.

Commitment and nullifier systems. Privacy-focused protocols such as Zcash Sapling [13] and Tornado Cash [14] employ commitment-based state models with nullifier-set replay protection. Semaphore [15] uses identity commitments for anonymous signaling. ZK-ACE draws on the commitment/nullifier design vocabulary but applies it to a distinct problem: *authorization* of transactions against a committed deterministic identity, rather than private value transfer or anonymous membership proofs.

Account abstraction and modular validation. ERC-4337 [16] introduces account abstraction for Ethereum, allowing accounts to define custom validation logic via validator modules. ZK-ACE can be instantiated as such a module, replacing signature-object checks with zero-knowledge proof verification. Unlike generic account-abstraction validators, ZK-ACE provides a complete authorization framework with formal security guarantees.

Identity-based cryptography. Identity-based encryption [19] and identity-based signatures [20] bind cryptographic capabilities to identity strings, typically requiring a trusted key-generation center. ZK-ACE does not assume any trusted third party; identity is self-sovereign and deterministically derived from user-controlled root material via a DIDP.

3 Preliminaries and Assumptions

3.1 Notation

We use the following notation throughout.

- λ : security parameter.
- $H(\cdot)$: a cryptographic hash function. When used inside zero-knowledge circuits, H is instantiated with a ZK-friendly hash (e.g., Poseidon [6]); outside circuits it may be a conventional hash.
- $\|$: concatenation with unambiguous parsing (e.g., length-prefixed encoding).
- **REV**: 256-bit Root Entropy Value serving as the identity root, derived via the DIDP. Ephemeral; never persistently stored.
- **Ctx**: derivation context tuple (**AlgID**, **Domain**, **Index**) used by the DIDP to derive context-specific keys from **REV**.
- **ID_{com}**: on-chain identity commitment.
- **TxHash**: hash of all authorization-relevant transaction fields.
- **domain**: chain or application domain separator.
- **target**: target-binding digest (address, call-data hash, or account context).
- **rp_{com}**: replay-prevention commitment (nonce commitment or nullifier).
- **PPT**: probabilistic polynomial-time.
- $\text{negl}(\lambda)$: a function negligible in λ .
- $x \xleftarrow{\$} S$: x sampled uniformly at random from set S .

3.2 Assumed DIDP Interface (Black-Box)

We assume the existence of a *deterministic identity derivation primitive* (DIDP), treated as a black-box building block. Rather than fixing a specific construction, we formalize only the interface and properties required by the zero-knowledge authorization layer. Any concrete framework satisfying this interface may be used; ACE-GF [1] is one such instantiation.

At the core of the DIDP is a high-entropy identity root called the *Root Entropy Value* (REV), a 256-bit value that uniquely defines the cryptographic identity of an entity and serves as the sole source of entropy for all derived keys. The REV exists only ephemerally in memory during execution and is never persistently stored or exported.

The DIDP provides two principal operations:

- **Identity reconstruction:** $\text{REV} \leftarrow \text{Unseal}(\text{params}, \text{SA}, \text{Cred})$, where SA is a sealed artifact (an encrypted encoding of REV) and Cred is an authorization credential. Returns REV if authentication succeeds, or $\perp$ otherwise. The credential controls *access* to the identity root but does not influence key derivation.
- **Context-specific key derivation:** $\text{Key} \leftarrow \text{Derive}(\text{REV}, \text{Ctx})$, where $\text{Ctx} = (\text{AlgID}, \text{Domain}, \text{Index})$ is a context descriptor encoding the target cryptographic algorithm (e.g., Ed25519, Secp256k1, ML-DSA), the application domain (e.g., signing, encryption), and a derivation index. The context tuple enforces cryptographic isolation: keys derived under distinct context tuples are computationally independent.

This two-operation interface cleanly separates the *authorization pipeline* (controlling access to REV via sealed artifacts and credentials) from the *identity pipeline* (deterministically deriving keys from REV using context-isolated derivation). The zero-knowledge authorization layer interacts only with the identity pipeline: it assumes the prover has obtained REV (via Unseal) and uses REV together with a context tuple Ctx to derive the relevant cryptographic material.

3.3 Security Properties Assumed from the DIDP

The zero-knowledge authorization layer relies on the following security properties of the assumed DIDP interface:

- **Determinism:** For fixed inputs, Derive produces identical outputs. In particular, the same (REV, Ctx) pair always yields the same derived key.
- **Context isolation:** Distinct context tuples $\text{Ctx} \neq \text{Ctx}'$ yield computationally independent derived keys. This ensures that compromise of a key derived under one context does not enable recovery of keys derived under other contexts, nor does it reveal information about the underlying REV .
- **Identity-root recovery hardness:** We formalize this property via the following game $\text{Game}_{\mathcal{A}}^{\text{rec-didp}}(\lambda)$:
 1. The challenger samples $\text{REV} \xleftarrow{\$} \{0, 1\}^{256}$.
 2. The adversary $\mathcal{A}$ is given oracle access to $\mathcal{O}_{\text{pub}}$, which on input $(\text{salt}, \text{domain})$ returns $H(\text{REV} \parallel \text{salt} \parallel \text{domain})$, and to $\mathcal{O}_{\text{derive}}$, which on input Ctx returns $H(\text{Derive}(\text{REV}, \text{Ctx}))$. These oracles model the ability to observe identity commitments and derived public identifiers without learning REV itself.
 3. $\mathcal{A}$ outputs a candidate REV^* .
 4. $\mathcal{A}$ wins if $\text{REV}^* = \text{REV}$.

For any PPT adversary $\mathcal{A}$:

$$\text{Adv}_{\mathcal{A}}^{\text{rec-didp}}(\lambda) = \Pr[\text{REV}^* = \text{REV}] \leq \text{negl}(\lambda).$$

Informally, no efficient adversary can recover the identity root from public outputs alone.

We note that identity-root recovery hardness is the *minimal* assumption required by the authorization soundness proof (Theorem 8.1). The proof’s case analysis (Section 8.1) shows that any successful forgery must either produce a hash collision (handled by collision resistance of H) or yield the correct REV from the extracted witness (handled by recovery hardness). A stronger notion—such as infeasibility of forging $H(\text{Derive}(\text{REV}, \text{Ctx}))$ for a fresh context Ctx without knowing REV —would follow from recovery hardness composed with the collision resistance of H , but is not needed as a separate assumption.

- **Multi-algorithm isolation:** Keys derived for different cryptographic algorithms (via distinct AlgID values in Ctx) from the same REV are computationally isolated, such that compromise of one derived key does not enable derivation of others.

No additional assumptions are made regarding the internal structure of the DIDP. In particular, this work does not depend on any specific key derivation function (e.g., HKDF [21]),

memory-hard password hashing scheme (e.g., Argon2 [5]), or mnemonic encoding used within the instantiation.

3.4 Zero-Knowledge Proof System Model

The constructions in this paper assume a general-purpose zero-knowledge proof system supporting succinct non-interactive arguments of knowledge, building on the foundational framework of interactive proofs [22]. The specific proving system is not fixed and may be instantiated using SNARK- [8], PLONK- [7], STARK- [9], or IPA-based constructions [10]. We assume the following standard properties:

- **Completeness:** An honest prover with a valid witness can always produce an accepting proof.
- **Knowledge soundness:** For any PPT prover $\mathcal{P}^*$ that produces an accepting proof with non-negligible probability, there exists a PPT extractor $\mathcal{E}$ that, given access to $\mathcal{P}^*$, outputs a valid witness with overwhelming probability. Formally,

$$\Pr[\text{Verify}(\text{vk}, \pi, x) = 1 \wedge \mathcal{E}^{\mathcal{P}^*}(x) \text{ fails}] \leq \text{negl}(\lambda).$$

- **Zero-knowledge:** Proofs reveal no information about the private witness beyond the validity of the statement.
- **Public-input binding:** The proof is cryptographically bound to the declared public inputs. For any PPT adversary,

$$\Pr[\text{Verify}(\text{vk}, \pi, x) = 1 \wedge \text{Verify}(\text{vk}, \pi, x') = 1 \wedge x \neq x'] \leq \text{negl}(\lambda).$$

- **Succinctness:** Proof size and verification complexity are independent of the size of the witness.

The precise extraction model depends on the chosen proof system: pairing-based SNARKs (e.g., Groth16) typically achieve knowledge soundness in the algebraic group model or via non-black-box extraction, while IOP-based systems (e.g., STARKs, PLONK) rely on point-query extraction from the underlying interactive oracle proof. Our security reductions are stated generically in terms of the knowledge-soundness advantage Adv^{ks} and are compatible with any proof system satisfying the above properties under its respective extraction model.

3.5 Threat Model

We consider an adversary $\mathcal{A}$ with the following capabilities:

- Full visibility into on-chain state, transaction history, and mempool-visible submissions.
- The ability to submit, reorder, replay, and substitute transaction payloads and attempt proof reuse across contexts, within the limits of the consensus protocol.
- The ability to perform offline cryptographic attacks, including attacks enabled by future quantum computation, against traditional public-key primitives.

The adversary may attempt replay attacks by reusing authorization artifacts, substitution attacks by binding valid authorization material to unintended transactions, or linkage attacks by correlating authorization data across transactions or contexts.

We do *not* assume the presence of trusted hardware, trusted execution environments, or private mempools. All authorization artifacts are assumed to be publicly observable. Compromise of the underlying DIDP root material (REV) for an identity is out of scope and treated as key compromise of the base primitive.

4 System Model

4.1 Roles

ZK-ACE involves the following roles:

- **Prover.** The prover is the entity controlling a deterministic identity derived via the DIDP, typically a user or wallet application. The prover generates zero-knowledge proofs attesting that a given transaction has been authorized by the corresponding identity.
- **Verifier.** The verifier is the consensus-facing component of the blockchain system, such as an on-chain smart contract or a native verification rule. The verifier checks the validity of submitted proofs and enforces replay protection according to protocol rules.
- **Optional aggregator or relayer.** In scalable deployments, an optional aggregator or relayer may collect authorization requests from multiple provers, aggregate proofs, or submit batched authorization data to the chain. This role is not trusted and does not learn private identity material.

The security of ZK-ACE does not depend on the honesty of aggregators or relayers; they serve only as performance and usability optimizations.

4.2 On-Chain State

ZK-ACE minimizes on-chain state by storing only data strictly required for authorization verification and replay prevention:

- **Identity commitment.** For each identity participating in ZK-ACE, the chain stores a compact commitment ID_{com} to the underlying deterministic identity. This commitment serves as the primary on-chain identity anchor for all future authorization proofs associated with that identity.
- **Replay protection state.** Depending on the chosen replay-prevention model (Section 7.4), the chain additionally maintains:
 - a *nonce registry*, enforcing monotonic or uniqueness constraints on authorization nonces; or
 - a *nullifier set*, marking authorization tokens as spent; or
 - an *authorization index* associated with the identity commitment.
- **Verifier parameters.** The verification key vk for the proof system.

No private keys, post-quantum signatures, or identity root material (REV) are stored on-chain.

4.3 Transaction Authorization Flow

The ZK-ACE authorization process for a single transaction proceeds as specified in Algorithm 1.

Throughout this process, no post-quantum signature objects are revealed or verified on-chain. Authorization validity is established solely through the correctness of the zero-knowledge proof and its binding to the committed identity and transaction hash.

5 Identity Commitment and Binding Model

5.1 Commitment Construction

For each deterministic identity participating in ZK-ACE, the chain stores a compact identity commitment defined as:

$$ID_{\text{com}} = H(\text{REV} \parallel \text{salt} \parallel \text{domain}), \quad (1)$$

where:

- REV is the 256-bit Root Entropy Value serving as the identity root, reconstructed ephemerally via the DIDP Unseal operation;
- $\text{salt} \xleftarrow{\$} \{0, 1\}^\lambda$ is an identity-specific random or pseudorandom value;
- domain encodes domain-separation parameters, such as the target blockchain or application namespace.

Algorithm 1 ZK-ACE Transaction Authorization

Prover side:

- 1: Construct transaction payload tx and compute $\text{TxHash} \leftarrow H(\text{tx})$.
- 2: Using identity material derived via the DIDP, generate witness $w = (\text{REV}, \text{salt}, \text{Ctx}, \text{nonce}, \text{aux})$.
- 3: Compute public inputs $\pi_{\text{pub}} = (\text{ID}_{\text{com}}, \text{TxHash}, \text{domain}, \text{target}, \text{rp_com})$.
- 4: Generate zero-knowledge proof $\pi \leftarrow \text{Prove}(\text{pk}, w, \pi_{\text{pub}})$.
- 5: Submit $(\text{tx}, \pi, \pi_{\text{pub}})$ to the chain (directly or via a relayer).

Verifier side (given $\text{tx}, \pi, \pi_{\text{pub}}$):

- 6: **Context binding check:** Recompute $\text{TxHash}' \leftarrow H(\text{tx})$ from the submitted transaction payload and verify $\text{TxHash}' = \pi_{\text{pub}}.\text{TxHash}$. Reject if mismatch.
 - 7: **Public-input validation:** Verify that $\pi_{\text{pub}}.\text{ID}_{\text{com}}$ is a registered on-chain identity commitment, and that $\pi_{\text{pub}}.\text{domain}$ and $\pi_{\text{pub}}.\text{target}$ match the expected values for the transaction context. Reject if any mismatch.
 - 8: **Proof verification:** Verify $\text{ZKVerify}(\text{vk}, \pi, \pi_{\text{pub}}) \stackrel{?}{=} 1$. Reject if verification fails.
 - 9: **Replay check:** Verify replay predicate (nonce freshness or nullifier novelty). Reject if violated.
 - 10: If all checks pass, accept transaction and update replay state.
-

The commitment serves as the primary on-chain identity anchor. Authorization additionally requires per-transaction public inputs (TxHash , domain , target , rp_com) and replay-prevention state, but ID_{com} is the persistent identity-binding component. Neither REV nor any derived keys are revealed on-chain.

The inclusion of salt serves multiple purposes. First, it enables *commitment re-randomization*, allowing the same underlying identity to be associated with multiple unlinkable commitments over time. Second, it provides a mechanism for *cross-chain or cross-domain isolation*, ensuring that identity commitments used in different execution environments are cryptographically distinct. Finally, salt-based randomization supports basic privacy controls by preventing trivial linkage between commitments derived from the same identity.

5.2 Public Input Binding Rules

Authorization proofs in ZK-ACE are verified against a set of public inputs that bind the proof to a specific transaction and execution context. These public inputs are consensus-visible and must be explicitly declared to the verifier. The following values are required:

1. **Transaction hash** (TxHash). Binds the authorization proof to a specific transaction payload, preventing substitution attacks.
2. **Chain or domain identifier** (domain). Ensures authorization proofs are valid only within the intended execution environment, preventing cross-chain replay.
3. **Derivation-target hash** (target). This is a hash commitment to context-derived authorization key material, i.e., $\text{target} = H(\text{Derive}(\text{REV}, \text{Ctx}))$, ensuring the proof is bound to a specific identity derivation context.
4. **Replay-prevention commitment** (rp_com). A nonce value, nullifier, or authorization-index commitment included to enforce single-use or monotonic authorization semantics.

Collectively, these public inputs ensure that each authorization proof is bound to *who* authorized the transaction (via ID_{com}), *which transaction* was authorized (via TxHash), *which derivation context* was used (via target), and *where and when* the authorization is valid (via domain and rp_com).

5.3 Linkability Considerations

ZK-ACE allows multiple identity commitments to be associated with the same underlying deterministic identity. This flexibility enables different trade-offs between privacy, usability, and auditability.

One approach is to maintain a single long-lived commitment per identity, maximizing simplicity and minimizing on-chain state. Alternatively, a prover may periodically rotate the salt value and register a new commitment, reducing linkability between authorization events at the cost of additional on-chain updates.

Importantly, linkability is controlled entirely at the commitment layer. Authorization proofs themselves reveal no additional information about the underlying identity beyond what is implied by the chosen commitment policy.

6 Authorization Semantics

6.1 What We Prove: Authorization

At a high level, ZK-ACE proves that a transaction is authorized by a committed identity under a deterministic derivation framework. The authorization statement proven in zero knowledge can be expressed as:

There exists a Root Entropy Value REV and associated derivation context such that:
(i) *the identity commitment derived from REV matches the on-chain commitment ID_{com} , and (ii) the identity anchored by REV has authorized the transaction with hash TxHash under the prescribed derivation context and replay-prevention rules.*

This statement is existential in nature. The zero-knowledge proof demonstrates the existence of a valid authorizing identity consistent with the on-chain commitment, without revealing REV or any derived keys. Authorization, in this model, is a *semantic property*: a transaction is valid if it can be shown that an identity anchored on-chain has approved the transaction according to the system’s deterministic authorization rules.

6.2 What We Do Not Prove

To avoid ambiguity, we explicitly state what ZK-ACE does *not* attempt to prove:

- **No verification of post-quantum signature objects.** The zero-knowledge circuit does not verify the correctness of any ML-DSA or other post-quantum signature. Signature objects are not provided as inputs to the circuit and do not appear on-chain.
- **No lattice arithmetic inside the circuit.** The circuit does not implement lattice-based cryptographic operations, nor does it encode the verification logic of any post-quantum signature scheme.
- **No compression of post-quantum signatures.** ZK-ACE does not claim to compress post-quantum signatures or reduce their size. Instead, it eliminates the need to place post-quantum signature material on-chain altogether.

The construction should therefore not be interpreted as a zero-knowledge optimization of post-quantum signatures. It represents a *shift in authorization semantics*, in which explicit signature verification is replaced by zero-knowledge proofs of deterministic authorization.

6.3 Authorization Token Definition

For concreteness, we define a unified authorization token that captures the semantic intent of transaction approval. Let the authorization token be:

$$\text{Auth} = H(\text{REV} \parallel \text{Ctx} \parallel \text{TxHash} \parallel \text{domain} \parallel \text{nonce}), \quad (2)$$

where each component binds the authorization to a specific identity root (REV), derivation context (Ctx), transaction (TxHash), execution domain (domain), and replay-prevention state (nonce).

The authorization token `Auth` is not a cryptographic signature and is not intended to be publicly verified outside the zero-knowledge proof. It serves as a conceptual representation of the authorization semantics enforced by the circuit. The zero-knowledge proof attests that such a token can be correctly derived from a committed identity root and that it binds the identity to the declared public inputs.

7 ZK Circuit Specification

This section specifies the zero-knowledge statement proven by ZK-ACE and the corresponding circuit structure. The circuit is designed to be compatible with standard constraint systems (e.g., R1CS or PLONK-like arithmetization) and to avoid embedding post-quantum signature verification logic.

7.1 Circuit Statement

Let ID_{com} be an on-chain identity commitment, $TxHash$ the transaction hash, $domain$ a chain/application domain tag, $target$ a derivation-target hash, and rp_{com} a replay-prevention commitment. The ZK-ACE circuit proves the following statement:

Prove knowledge of $(REV, salt, Ctx, nonce, aux)$ such that:

1. *the identity commitment recomputed from $(REV, salt, domain)$ equals the public commitment ID_{com} ;*
2. *the target binding is consistent with deterministic key derivation under (REV, Ctx) ;*
3. *the transaction hash $TxHash$ is authorized under the derived context and replay-prevention rule.*

This statement is existential: the proof establishes that *some* identity root consistent with ID_{com} authorized the specific transaction context, without revealing REV or any derived keys.

7.2 Witness and Public Inputs

Private witness (secret inputs). The witness consists of:

- **REV:** the 256-bit Root Entropy Value (identity root, reconstructed ephemerally via DIDP Unseal);
- **salt:** commitment salt associated with the identity;
- **Ctx:** derivation context tuple ($AlgID, Domain, Index$) specifying the target cryptographic algorithm, application domain, and derivation index;
- **nonce:** replay-prevention value (or material used to derive a nullifier);
- **aux:** auxiliary secret material that may be required by a specific circuit instantiation (e.g., intermediate absorption states for multi-round Poseidon calls, or padding values for variable-length input encoding). The core constraints (C1)–(C5) are stated without reference to **aux**; it is included in the witness definition as an instantiation-reserved slot so that concrete deployments can carry implementation-specific intermediates without modifying the abstract witness structure.

Public inputs. The public inputs consist of:

- **ID_{com} :** on-chain identity commitment;
- **$TxHash$:** transaction hash to be authorized;
- **domain:** chain/application domain tag;

- **target**: hash commitment to the context-derived authorization key material, i.e., $\text{target} = H(\text{Derive}(\text{REV}, \text{Ctx}))$;
- **rp_com**: replay-prevention commitment (nonce commitment or nullifier).

All public inputs are consensus-visible and must be bound to the proof to prevent substitution and replay attacks.

7.3 Core Constraints

We describe the circuit constraints at a high level, in a form intended to map naturally to standard arithmetization. Let $H(\cdot)$ denote the selected in-circuit hash used for binding.

- (C1) Commitment consistency.** The circuit recomputes the identity commitment and enforces equality with ID_{com} :

$$H(\text{REV} \parallel \text{salt} \parallel \text{domain}) = \text{ID}_{\text{com}}. \quad (3)$$

This ensures that the prover controls an identity root consistent with the on-chain anchor.

- (C2) Deterministic derivation correctness (target binding).** The circuit enforces that the public target hash corresponds to the context-specific key derived from the identity root:

$$\text{target} = H(\text{Derive}(\text{REV}, \text{Ctx})). \quad (4)$$

The outer hash ensures that the derived key itself is never exposed as a public input; only a hash commitment to it (**target**) is revealed on-chain, preserving the secrecy of identity-derived cryptographic material. The domain component of Ctx is constrained to equal the public input **domain** by constraint (C5), ensuring that derivation correctness is bound to the declared domain.

Implementation note: To minimize constraint count, **Derive** is instantiated inside the circuit as a deterministic sequence of ZK-friendly hashes (e.g., Poseidon), avoiding the overhead of simulating non-native field arithmetic or expensive conventional encodings (e.g., Bech32, Base58) inside the circuit. The context tuple $\text{Ctx} = (\text{AlgID}, \text{Domain}, \text{Index})$ is encoded as a fixed-length field-element input to the hash.

Instantiation scope: The function **Derive** referenced in (C2) denotes the *circuit-native derivation function*—i.e., the concrete ZK-friendly hash sequence evaluated inside the proof system—rather than the external DIDP **Derive** implementation, which may use non-circuit-friendly primitives (e.g., HKDF, HMAC-SHA-256). The security analysis (Section 8) operates entirely on the circuit-native function: what the proof system enforces is that the prover knows a **REV** from which the declared **target** can be derived via this circuit-native function. Ensuring *functional equivalence* between the circuit-native derivation and the off-chain DIDP derivation—i.e., that both produce the same output on identical (REV, Ctx) inputs—is a deployment-time instantiation requirement. It is satisfied whenever the off-chain DIDP pipeline adopts the same hash function and encoding convention used in the circuit (a natural choice, since the ZK-friendly hash is equally valid off-chain). This separation follows standard practice in ZK protocol design, where the circuit defines the “canonical” computation and external components are aligned to it.

- (C3) Authorization binding to TxHash.** The circuit binds the authorization to the specific transaction hash and context by enforcing the authorization token relation:

$$\text{Auth} = H(\text{REV} \parallel \text{Ctx} \parallel \text{TxHash} \parallel \text{domain} \parallel \text{nonce}). \quad (5)$$

(C4) Anti-replay (nonce/nullifier rule). The circuit commits to replay-prevention state in a way that the verifier can enforce uniqueness or monotonicity. Two canonical forms are supported:

- *Nonce commitment:*

$$\text{rp_com} = H(\text{ID}_{\text{com}} \parallel \text{nonce}), \quad (6)$$

enabling the chain to enforce per-identity nonce rules.

- *Nullifier (single-use token):*

$$\text{rp_com} = H(\text{Auth} \parallel \text{domain}), \quad (7)$$

enabling the chain to reject reuse by maintaining a nullifier set.

(C5) Domain separation and context consistency. The circuit enforces that the domain value declared as a public input is used consistently in all commitment and binding computations, preventing cross-chain or cross-application proof reuse. This includes two requirements:

- Public-input consistency:* `domain` appears as an explicit input to the commitment binding (C1), the authorization binding (C3), and the replay-prevention computation (C4).
- Context-domain binding:* The circuit enforces that the domain component of the context tuple matches the publicly declared domain:

$$\text{Ctx.Domain} = \text{domain}. \quad (8)$$

This constraint ensures that the derivation in (C2) operates under the same domain as all other bindings, closing the link between the context-specific derivation and the public domain identifier.

These constraints ensure that a valid proof simultaneously attests to identity ownership (via preimage knowledge of `REV` for `IDcom`), context correctness (via deterministic derivation under `Ctx`), transaction-specific authorization (via binding to `TxHash`), and replay prevention (via `rp_com`).

Constraint count estimation. The circuit cost is dominated by the in-circuit hash evaluations. Table 1 provides the measured constraint breakdown from the reference implementation using Poseidon with width $t = 3$, S-box exponent $\alpha = 17$, 8 full rounds, and 57 partial rounds over the BN254 scalar field.²

The per-invocation cost scales with input arity: the arkworks sponge gadget (rate $r = 2$) requires $\lceil k/r \rceil$ absorption rounds for k field-element inputs, at ≈ 400 R1CS constraints per round. (C2) requires two hash invocations—an inner hash over 4 inputs plus an outer hash over 1 input. (C3) absorbs 7 field elements (`REV` $\parallel$ `AlgID` $\parallel$ `Domain` $\parallel$ `Index` $\parallel$ `TxHash` $\parallel$ `domain` $\parallel$ `nonce`), requiring 4 absorption rounds and accounting for 40% of the total circuit.

Both replay modes (nonce-registry and nullifier-set) produce identical constraint counts. Domain separation (C5) is enforced as an equality constraint on the domain component of `Ctx` and adds negligible overhead.

For comparison, verifying a lattice-based signature such as ML-DSA inside a zero-knowledge circuit requires expressing: (i) Number Theoretic Transforms (NTTs) over degree-256 polynomial rings in $\mathbb{Z}_q$ with $q = 2^{23} - 2^{13} + 1$, each requiring $O(n \log n)$ multiplications in a non-native field; (ii) high-bit-width modular reductions (23-bit modulus emulated over a ≈ 254 -bit

²The original version of this paper cited the Poseidon paper’s [6] theoretical estimate of ≈ 250 – 300 constraints per hash invocation, which assumes $\alpha = 5$ and fewer partial rounds. The reference implementation uses $\alpha = 17$ with 57 partial rounds for the BN254 scalar field.

Table 1: Measured constraint breakdown for the ZK-ACE circuit (Poseidon, $t = 3$, $\alpha = 17$, BN254, arkworks).

Constraint	Inputs	Hash calls	R1CS
(C1) Commitment consistency	3	1	805
(C2) Derivation correctness	$4 + 1$	2	1,200
(C3) Authorization binding	7	1	1,615
(C4) Replay prevention	2	1	400
(C5) Domain sep. + <code>enforce_equal</code>	—	—	4
Total		5	4,024

proof-system field, requiring range-check gadgets per limb); and (iii) rejection-sampling and hint-reconstruction logic with data-dependent branching. ML-DSA verification at NIST level 2 involves $k \times \ell = 4 \times 4$ matrix-vector NTTs and $k = 4$ polynomial reconstructions, yielding a structural lower bound on the order of millions of R1CS constraints even with optimized non-native arithmetic. The measured ZK-ACE circuit (4,024 R1CS constraints; Table 1) is therefore **roughly three orders of magnitude smaller** than an in-circuit post-quantum signature verification. This gap reflects the structural difference between a small number of ZK-friendly hash invocations and the full algebraic machinery of lattice-based signature verification. The resulting circuit size makes proving feasible on consumer-grade hardware (≈ 63 ms single-threaded; see Section 12) and enables practical batch and recursive composition (Section 11).

7.4 Replay Protection Designs

ZK-ACE supports multiple replay-prevention strategies. We present two canonical options.

Option A: Nonce registry (account-style model). In an account-style model, each identity commitment maintains a nonce state on-chain. The proof includes a nonce commitment (Eq. 6) as a public input, and the verifier enforces that the nonce is fresh according to the protocol rule (e.g., strictly increasing, or equal to the expected next nonce).

Trade-offs. Nonce registries provide simple, deterministic replay protection and integrate naturally with account abstraction. However, they introduce per-identity state updates and reveal ordering information, which may be undesirable in privacy-preserving settings. Additionally, nonce-based models require sequential submission, limiting parallelism.

Option B: Nullifier set (privacy-style model). In a privacy-oriented model, the proof outputs a nullifier derived from the authorization token (Eq. 7). The verifier maintains a global set of spent nullifiers and rejects any transaction whose nullifier has appeared before.

Trade-offs. Nullifiers support single-use authorization without requiring a per-identity nonce counter and can better accommodate unlinkability across authorizations. However, maintaining and checking a nullifier set introduces global state growth, and careful design is required to prevent unintended linkage across domains.

Remark 7.1. *The choice between nonce and nullifier models is deployment-specific and does not affect the core security properties of ZK-ACE. Both models satisfy the replay-resistance definition (Definition 8.2) under their respective verifier enforcement rules.*

8 Security Definitions and Analysis

This section formalizes the security properties of ZK-ACE via explicit game-based definitions and provides reduction-based proofs to standard assumptions, following the code-based game-

playing methodology [23]. For a comprehensive treatment of the underlying cryptographic tools, we refer the reader to [24]. Throughout, $\mathcal{A}$ denotes a PPT adversary.

Remark 8.1 (Mapping practical attack scenarios to formal games). *The threat model (Section 3.5) grants $\mathcal{A}$ the ability to submit, reorder, replay, and substitute transaction payloads. The four security games defined below capture these capabilities as follows:*

- *Unauthorized authorization (identity theft): An adversary who does not possess REV attempts to produce a valid proof for an on-chain identity commitment. This is captured by the authorization soundness game (Definition 8.1).*
- *Front-running and transaction substitution: An adversary observes a valid authorization proof in the mempool and attempts to rebind it to a different transaction hash. Because the proof is cryptographically bound to TxHash via constraint (C3), this is captured by the substitution-resistance game (Definition 8.3).*
- *Replay and state-race attacks: An adversary attempts to resubmit a previously accepted authorization, or to exploit concurrent submissions to cause double-spending. Both are captured by the replay-resistance game (Definition 8.2), whose analysis covers the nonce (sequential ordering) and nullifier (set-membership) enforcement models.*
- *Cross-chain or cross-application proof migration: An adversary takes a valid proof from one domain and attempts to use it in another. This is captured by the cross-domain separation game (Definition 8.4), enforced by the inclusion of domain in all commitment and binding computations (C1–C5).*

Together, the four games provide coverage of the attack surface described in the threat model. Attacks not captured—such as compromise of the underlying REV or denial-of-service at the consensus layer—are out of scope, as noted in Section 3.5.

8.1 Authorization Soundness

Definition 8.1 (Authorization Soundness Game). *The game $\text{Game}_{\mathcal{A}}^{\text{auth}}(\lambda)$ proceeds as follows:*

1. **Setup.** *The challenger generates a fresh identity: $\text{REV} \xleftarrow{\$} \{0,1\}^{256}$, $\text{salt} \xleftarrow{\$} \{0,1\}^{\lambda}$, and $\text{ID}_{\text{com}} = H(\text{REV} \parallel \text{salt} \parallel \text{domain})$. The verifier key vk and the commitment ID_{com} are given to $\mathcal{A}$. The values REV and salt are not disclosed.*
2. **Oracle access.** *$\mathcal{A}$ may observe all public on-chain data, including previously accepted proofs and their public inputs.*
3. **Output.** *$\mathcal{A}$ outputs a candidate $(\pi^*, \pi_{\text{pub}}^*)$ where π_{pub}^* includes ID_{com} as the identity commitment.*
4. **Win condition.** *$\mathcal{A}$ wins if $\text{ZKVerify}(\text{vk}, \pi^*, \pi_{\text{pub}}^*) = 1$ and the replay predicate passes for the submitted tuple.*

The advantage of $\mathcal{A}$ is $\text{Adv}_{\mathcal{A}}^{\text{auth}}(\lambda) = \Pr[\text{Game}_{\mathcal{A}}^{\text{auth}}(\lambda) = 1]$.

Remark 8.2 (Strength of the adversarial model). *Definition 8.1 uses passive observation of accepted proofs as the baseline adversarial interface. This model is sufficient for our core reductions. Extending to an explicit adaptive chosen-message authorization oracle is possible, but requires additional proof-system-level assumptions (e.g., simulation soundness/non-malleability of the authorization interface) to preserve an unchanged bound. We leave that strengthening as future formalization.*

Theorem 8.1 (Authorization Soundness). *Under (i) knowledge soundness of the proof system, (ii) collision resistance of H , and (iii) identity-root recovery hardness of the DIDP (Section 3.3), for any PPT adversary $\mathcal{A}$:*

$$\text{Adv}_{\mathcal{A}}^{\text{auth}}(\lambda) \leq \text{Adv}_{\mathcal{B}_1}^{\text{ks}}(\lambda) + \text{Adv}_{\mathcal{B}_2}^{\text{cr}}(\lambda) + \text{Adv}_{\mathcal{B}_3}^{\text{rec-didp}}(\lambda) + \text{negl}(\lambda).$$

Proof. Suppose $\mathcal{A}$ outputs $(\pi^*, \pi_{\text{pub}}^*)$ that the verifier accepts. We construct reductions $\mathcal{B}_1, \mathcal{B}_2, \mathcal{B}_3$ as follows.

Step 1: Witness extraction. By the knowledge-soundness property of the proof system, there exists a PPT extractor $\mathcal{E}$ such that, given $\mathcal{A}$'s accepting proof and randomness, $\mathcal{E}$ outputs a witness $w^* = (\text{REV}^*, \text{salt}^*, \text{Ctx}^*, \text{nonce}^*, \text{aux}^*)$ satisfying all circuit constraints (C1)–(C5), except with probability at most $\text{Adv}_{\mathcal{B}_1}^{\text{ks}}(\lambda)$. We construct $\mathcal{B}_1$ that simulates the game for $\mathcal{A}$ and, upon receiving an accepting proof, invokes $\mathcal{E}$ to recover the witness.

Step 2: Case analysis on the extracted witness. Assume extraction succeeds. By constraint (C1), the extracted witness satisfies:

$$H(\text{REV}^* \parallel \text{salt}^* \parallel \text{domain}) = \text{ID}_{\text{com}} = H(\text{REV} \parallel \text{salt} \parallel \text{domain}).$$

We consider two cases:

Case 1: $(\text{REV}^*, \text{salt}^*) \neq (\text{REV}, \text{salt})$. In this case, $\mathcal{A}$ has produced two distinct preimages of ID_{com} under H , constituting a collision. We construct $\mathcal{B}_2$ that embeds a collision-resistance challenge into H and uses $\mathcal{A}$ as a subroutine: $\mathcal{B}_2$ simulates the game honestly using the challenge hash, and when extraction yields $(\text{REV}^*, \text{salt}^*) \neq (\text{REV}, \text{salt})$, outputs both preimages as a collision. Thus $\text{Adv}_{\mathcal{B}_2}^{\text{cr}}(\lambda) \geq \Pr[\text{Case 1}]$.

Case 2: $\text{REV}^* = \text{REV}$ and $\text{salt}^* = \text{salt}$. In this case, the extracted witness contains the correct REV . Since $\mathcal{A}$ was not given REV , this constitutes a breach of DIDP identity-root recovery hardness (Section 3.3). We construct $\mathcal{B}_3$ as follows. $\mathcal{B}_3$ participates in the recovery game $\text{Game}_{\text{rec-didp}}$, receiving oracle access to $\mathcal{O}_{\text{pub}}$ and $\mathcal{O}_{\text{derive}}$ from the DIDP challenger. $\mathcal{B}_3$ samples $\text{salt} \xleftarrow{\$} \{0, 1\}^\lambda$ and queries $\mathcal{O}_{\text{pub}}(\text{salt}, \text{domain})$ to obtain $\text{ID}_{\text{com}} = H(\text{REV} \parallel \text{salt} \parallel \text{domain})$ without learning REV . Similarly, $\mathcal{B}_3$ queries $\mathcal{O}_{\text{derive}}(\text{Ctx})$ to obtain the target-binding value $H(\text{Derive}(\text{REV}, \text{Ctx}))$ for any context needed to simulate public on-chain data. Using ID_{com} and the derived public values, $\mathcal{B}_3$ faithfully simulates the authorization soundness game for $\mathcal{A}$. When extraction succeeds and yields $\text{REV}^* = \text{REV}$, $\mathcal{B}_3$ outputs REV^* as its answer in the DIDP recovery game. Thus $\text{Adv}_{\mathcal{B}_3}^{\text{rec-didp}}(\lambda) \geq \Pr[\text{Case 2}]$.

Step 3: Combining. By a union bound over all failure events:

$$\text{Adv}_{\mathcal{A}}^{\text{auth}}(\lambda) \leq \text{Adv}_{\mathcal{B}_1}^{\text{ks}}(\lambda) + \text{Adv}_{\mathcal{B}_2}^{\text{cr}}(\lambda) + \text{Adv}_{\mathcal{B}_3}^{\text{rec-didp}}(\lambda) + \text{negl}(\lambda). \quad \square$$

8.2 Replay Resistance

Definition 8.2 (Replay-Resistance Game). *The game $\text{Game}_{\mathcal{A}}^{\text{replay}}(\lambda)$ proceeds as follows:*

1. **Setup.** *As in Definition 8.1. The honest prover produces one valid authorization tuple $(\pi_1, \pi_{1,\text{pub}})$ for a transaction with hash TxHash_1 , which is accepted and applied to the verifier state (nonce incremented or nullifier inserted).*
2. **Oracle access.** *$\mathcal{A}$ receives $(\pi_1, \pi_{1,\text{pub}})$ and full access to on-chain state.*
3. **Win condition.** *$\mathcal{A}$ wins if it causes acceptance of a second authorization tuple $(\pi^*, \pi_{\text{pub}}^*)$ where π_{pub}^* includes ID_{com} and the same logical authorization event (i.e., the tuple violates the protocol's replay-prevention invariant).*

Theorem 8.2 (Replay Resistance). *Under the assumptions of Theorem 8.1 and correct verifier enforcement of the replay predicate, for any PPT adversary $\mathcal{A}$:*

$$\text{Adv}_{\mathcal{A}}^{\text{replay}}(\lambda) \leq \text{Adv}_{\mathcal{A}}^{\text{auth}}(\lambda) + \text{negl}(\lambda).$$

Proof. We consider each replay model:

Nonce model. After the honest authorization with nonce n , the verifier state advances to expect nonce $n + 1$.

- *Direct replay:* Resubmitting $(\pi_1, \pi_{1,\text{pub}})$ produces rp_com computed with nonce n . The verifier rejects because the expected nonce is $n + 1$.

- *New proof with stale nonce:* Any new proof with rp_com computed using $\text{nonce} \leq n$ is rejected by the monotonicity check.
- *New proof with fresh nonce:* Producing a valid proof with $\text{nonce } n+1$ requires a valid witness satisfying (C1)–(C5), which reduces to the authorization soundness game (Theorem 8.1).

Nullifier model. After the honest authorization, the nullifier $\text{rp_com}_1 = H(\text{Auth}_1 \parallel \text{domain})$ is inserted into the nullifier set $\mathcal{N}$.

- *Direct replay:* Resubmitting $(\pi_1, \pi_{1,\text{pub}})$ produces the same $\text{rp_com}_1 \in \mathcal{N}$, which is rejected.
- *Same authorization, different proof:* Any proof for the same $(\text{REV}, \text{Ctx}, \text{TxHash}, \text{domain}, \text{nonce})$ produces the same Auth_1 and hence the same nullifier, which is rejected.
- *Different authorization parameters:* Changing any component of the Auth computation yields a different Auth' and a fresh nullifier $\text{rp_com}' \notin \mathcal{N}$. This represents a genuinely new authorization event (different transaction, context, or nonce) and is accepted—correctly, as it is not a replay.

In both models, the only way for $\mathcal{A}$ to cause acceptance of a replayed authorization event is to produce a valid proof with a fresh replay token, which requires knowledge of a valid witness. This reduces to authorization soundness. $\square$

8.3 Substitution Resistance

Definition 8.3 (Substitution-Resistance Game). *The game $\text{Game}_{\mathcal{A}}^{\text{subst}}(\lambda)$ proceeds as follows:*

1. **Setup.** *As in Definition 8.1. An honest authorization tuple (π, π_{pub}) is generated for public inputs $(\text{ID}_{\text{com}}, \text{TxHash}, \text{domain}, \text{target}, \text{rp_com})$.*
2. **Oracle access.** *$\mathcal{A}$ receives (π, π_{pub}) .*
3. **Win condition.** *$\mathcal{A}$ wins if the verifier accepts (π, π'_{pub}) where $\pi'_{\text{pub}} \neq \pi_{\text{pub}}$, i.e., the same proof is accepted with modified public inputs (different TxHash , target , or domain).*

Theorem 8.3 (Substitution Resistance). *Under the public-input binding property of the proof system, for any PPT adversary $\mathcal{A}$:*

$$\text{Adv}_{\mathcal{A}}^{\text{subst}}(\lambda) \leq \text{Adv}_{\mathcal{B}}^{\text{pib}}(\lambda) + \text{negl}(\lambda).$$

Proof. Suppose $\mathcal{A}$ outputs $\pi'_{\text{pub}} \neq \pi_{\text{pub}}$ such that $\text{ZKVerify}(\text{vk}, \pi, \pi'_{\text{pub}}) = 1$. Then the same proof π verifies under two distinct public input tuples. We construct $\mathcal{B}$ that embeds this as a public-input binding challenge: $\mathcal{B}$ simulates the substitution game for $\mathcal{A}$, and upon $\mathcal{A}$'s success, outputs $(\pi, \pi_{\text{pub}}, \pi'_{\text{pub}})$ as a violation of public-input binding.

By the public-input binding property (Section 3.4), this occurs with at most negligible probability, giving:

$$\text{Adv}_{\mathcal{A}}^{\text{subst}}(\lambda) \leq \text{Adv}_{\mathcal{B}}^{\text{pib}}(\lambda) + \text{negl}(\lambda). \quad \square$$

8.4 Cross-Domain Separation

Definition 8.4 (Cross-Domain Separation Game). *The game $\text{Game}_{\mathcal{A}}^{\text{domain}}(\lambda)$ proceeds as follows:*

1. **Setup.** *The challenger registers the same identity on two distinct domains $\text{domain}_1 \neq \text{domain}_2$ with respective commitments $\text{ID}_{\text{com}}^{(1)} = H(\text{REV} \parallel \text{salt}_1 \parallel \text{domain}_1)$ and $\text{ID}_{\text{com}}^{(2)} = H(\text{REV} \parallel \text{salt}_2 \parallel \text{domain}_2)$, where $\text{salt}_1, \text{salt}_2$ are independently sampled. An honest authorization proof (π, π_{pub}) is generated for domain domain_1 .*
2. **Win condition.** *$\mathcal{A}$ wins if it causes acceptance of an authorization tuple on domain domain_2 using material from the domain- domain_1 authorization (either by proof reuse or derivation).*

Theorem 8.4 (Cross-Domain Separation). *Under collision resistance of H and public-input binding of the proof system, for any PPT adversary $\mathcal{A}$:*

$$\text{Adv}_{\mathcal{A}}^{\text{domain}}(\lambda) \leq \text{Adv}_{\mathcal{B}_1}^{\text{pib}}(\lambda) + \text{Adv}_{\mathcal{B}_2}^{\text{cr}}(\lambda) + \text{negl}(\lambda).$$

Proof. We consider two attack strategies:

Proof reuse. If $\mathcal{A}$ submits the original proof π (generated for domain domain_1) to the domain- domain_2 verifier, the public inputs must include $\text{ID}_{\text{com}}^{(2)}$ and domain_2 . Since π was generated for public inputs containing $\text{ID}_{\text{com}}^{(1)}$ and domain_1 , and $(\text{ID}_{\text{com}}^{(1)}, \text{domain}_1) \neq (\text{ID}_{\text{com}}^{(2)}, \text{domain}_2)$, verification fails by public-input binding.

Commitment collision. If $\mathcal{A}$ attempts to find $(\text{REV}', \text{salt}', \text{domain}_2)$ such that $H(\text{REV}' \parallel \text{salt}' \parallel \text{domain}_2) = \text{ID}_{\text{com}}^{(1)}$, this yields two distinct domain-tagged inputs mapping to the same hash value and therefore constitutes a collision event under H .

Moreover, by constraint (C5), domain is included in all circuit bindings. A proof generated with $\text{domain} = \text{domain}_1$ cannot satisfy the verifier's constraint checks when the declared domain is domain_2 , as the commitment, authorization token, and replay-prevention commitment all incorporate domain . Combining: $\text{Adv}_{\mathcal{A}}^{\text{domain}}(\lambda) \leq \text{Adv}_{\mathcal{B}_1}^{\text{pib}}(\lambda) + \text{Adv}_{\mathcal{B}_2}^{\text{cr}}(\lambda) + \text{negl}(\lambda)$. $\square$

8.5 Composition

Corollary 8.1 (Combined Security). *Under the assumptions of Theorems 8.1–8.4, ZK-ACE simultaneously satisfies authorization soundness, replay resistance, substitution resistance, and cross-domain separation against PPT adversaries. Specifically, for any PPT adversary $\mathcal{A}$ attacking any single property, there exist PPT reductions $\mathcal{B}_1, \dots, \mathcal{B}_5$ such that:*

$$\text{Adv}_{\mathcal{A}}^{\text{combined}}(\lambda) \leq \text{Adv}_{\mathcal{B}_1}^{\text{ks}}(\lambda) + 2 \cdot \text{Adv}_{\mathcal{B}_2}^{\text{cr}}(\lambda) + \text{Adv}_{\mathcal{B}_3}^{\text{rec-didp}}(\lambda) + 2 \cdot \text{Adv}_{\mathcal{B}_4}^{\text{pib}}(\lambda) + \text{negl}(\lambda),$$

where the factor of 2 on Adv^{cr} arises because both the authorization soundness proof (Theorem 8.1, Case 1) and the cross-domain separation proof (Theorem 8.4) each reduce to collision resistance, and the factor of 2 on Adv^{pib} arises because both the substitution resistance proof (Theorem 8.3) and the cross-domain separation proof each reduce to public-input binding. The bound is negligible under the stated assumptions.

Proposition 8.1 (Completeness). *If the honest prover holds a valid witness $w = (\text{REV}, \text{salt}, \text{Ctx}, \text{nonce}, \text{aux})$ satisfying all constraints (C1)–(C5), and the replay-prevention state permits the submitted token (i.e., nonce is fresh or nullifier is unspent), then the verifier accepts the resulting proof with probability $1 - \text{negl}(\lambda)$.*

Proof. By the completeness property of the underlying proof system, an honestly generated proof for a satisfying witness is accepted with overwhelming probability. The replay-prevention predicate passes by hypothesis on the freshness of the submitted nonce or nullifier. All public inputs are computed consistently with the witness by construction, so the verifier's constraint checks are satisfied. $\square$

Remark 8.3 (Privacy Guarantee). *By the zero-knowledge property of the underlying proof system, ZK-ACE authorization proofs reveal no information about the private witness $(\text{REV}, \text{salt}, \text{Ctx}, \text{nonce}, \text{aux})$ beyond what is implied by the public inputs $(\text{ID}_{\text{com}}, \text{TxHash}, \text{domain}, \text{target}, \text{rp_com})$. In particular:*

- The 256-bit Root Entropy Value REV remains hidden. Since REV is the sole identity root from which all cryptographic material is derived, its concealment ensures that no derived keys can be computed by observers.
- The commitment salt salt is not disclosed, preserving identity unlinkability across commitments derived with different salts.
- The derivation context Ctx and nonce nonce are hidden, preventing observers from inferring the target algorithm, application domain, derivation index, or replay-prevention state beyond what the public rp_com reveals.

The information revealed to observers is limited to: (i) which identity commitment was used, (ii) which transaction was authorized, (iii) the domain and target context, and (iv) the replay-prevention token. This leakage profile is inherent to any authorization system that must bind proofs to specific transactions and enforce replay prevention, and represents the minimal information disclosure required for consensus verification.

9 On-Chain Verification and Deployment

9.1 Verification Interface

At the consensus layer, ZK-ACE exposes a minimal verification interface that accepts a zero-knowledge proof together with a fixed set of public inputs. Conceptually, the verifier evaluates a predicate of the form:

$$\text{ZKVerify}(\text{proof}, \text{public_inputs}) \in \{\text{accept}, \text{reject}\}.$$

Crucially, the verifier does not accept the prover’s declared public inputs at face value. Before evaluating the proof, the verifier must independently verify context consistency (Algorithm 1, steps 6–7): (i) recompute TxHash from the submitted transaction payload and confirm it matches the value declared in π_{pub} ; (ii) confirm that ID_{com} corresponds to a registered on-chain identity commitment; and (iii) confirm that **domain** and **target** are consistent with the expected transaction context. Only after these checks pass does the verifier evaluate the zero-knowledge proof and, upon successful verification, perform any required state updates associated with replay prevention:

- recording the consumed nonce in a per-identity nonce registry;
- inserting the nullifier into a global nullifier set to prevent reuse; or
- updating an authorization index or similar monotonic state variable.

These updates are deterministic and verifiable by all consensus participants. No identity root material (REV) or derived keys are ever written to chain state.

9.2 Compatibility with Account Abstraction

ZK-ACE is naturally compatible with account-abstraction (AA) architectures, including designs aligned with ERC-4337-style transaction flows [16].

Validator module integration. In an AA setting, ZK-ACE can be implemented as a *validator module* associated with an abstract account. Instead of checking a classical or post-quantum signature, the account validation logic invokes the ZK-ACE verification interface to determine whether the transaction is authorized by the committed identity.

Bundler and submission flow. The generation of zero-knowledge proofs occurs off-chain and is performed by the entity controlling the identity, typically the user’s wallet software. A bundler or relayer may collect user operations and submit them to the chain, but does not need access to any secret identity material. The bundler merely transports the proof and public inputs and has no influence over authorization correctness. This separation preserves the trust-minimized design of account abstraction while allowing proof generation and transaction submission responsibilities to be flexibly allocated.

9.3 Instantiation Flexibility

Proof system selection. ZK-ACE may be instantiated using different classes of zero-knowledge proof systems. SNARK-based systems (e.g., Groth16 [8]) offer the smallest proof sizes and

fastest verification, at the cost of a trusted setup. Universal systems (e.g., PLONK [7]) avoid per-circuit trusted setup. Transparent systems (e.g., STARKs [9]) require no trusted setup at all but produce larger proofs. IPA-based systems (e.g., Bulletproofs [10]) offer a different trade-off between proof size and verification time. The framework does not rely on properties unique to any particular system beyond soundness, zero-knowledge, and public-input binding.

Hash function parameterization. ZK-friendly hash functions, such as Poseidon [6], are used within the circuit for commitments and bindings. Parameter choices (e.g., arity, round numbers, field size) are left to the deployment context. The security arguments of ZK-ACE rely only on standard collision and preimage resistance assumptions, not on any specific parameter set.

Proof-system migration. Because identity commitments are proof-system agnostic (they depend only on the hash function, not the ZK proof system), verifier and prover upgrades do not require identity rotation. A chain can migrate from one proof system to another without invalidating existing identity commitments.

10 Structural On-Chain Data Accounting

This section provides a structural accounting of on-chain authorization data under different authorization models. The purpose is to compare the *protocol-imposed data artifacts* required by post-quantum signature-based authorization and by ZK-ACE authorization, independent of concrete implementations or performance benchmarks.

10.1 Accounting Methodology

The analysis is not based on experimental measurements, runtime benchmarks, or gas profiling. Instead, it relies on protocol-level accounting of the data objects that must be included on-chain for transaction authorization under each model. Specifically, we compare:

- the mandatory authorization artifacts that consensus participants must receive, verify, and store; and
- the asymptotic size of these artifacts as dictated by cryptographic standards and protocol structure.

10.2 Baseline: Post-Quantum Signature Artifacts

In a post-quantum signature-based authorization model, each transaction must carry a cryptographic signature object that is verified by the consensus layer. For lattice-based signature schemes, standardized parameter sets result in signature sizes on the order of several kilobytes. Table 2 summarizes representative sizes.

The defining characteristic is that post-quantum signature objects are *kilobyte-scale*. These objects must be transmitted, verified, and permanently recorded as part of the ledger, making their size a first-order contributor to on-chain data growth.

10.3 ZK-ACE Authorization Artifacts

Under ZK-ACE, explicit post-quantum signature objects are not placed on-chain. Instead, authorization is represented by a combination of compact commitments and succinct zero-knowledge proofs. The required on-chain artifacts per transaction are:

- **Identity commitment:** A hash-based commitment to the deterministic identity (ID_{com}), typically occupying 32 bytes.

Table 2: Post-quantum signature artifact sizes for standardized schemes.

Scheme	NIST Level	Signature Size	Public Key
ML-DSA-44 [2]	2	2,420 bytes	1,312 bytes
ML-DSA-65 [2]	3	3,309 bytes	1,952 bytes
ML-DSA-87 [2]	5	4,627 bytes	2,592 bytes
SLH-DSA-128f [3]	1	17,088 bytes	32 bytes
FN-DSA-512 [4]	1	$\approx$ 666 bytes	897 bytes
Ed25519 (classical)		64 bytes	32 bytes
secp256k1 ECDSA (classical)		$\approx$ 71 bytes	33 bytes

Table 3: Structural comparison of per-transaction authorization data. PQC column assumes first-use public-key transmission (amortized to zero for repeat senders in practice). ZK-ACE column assumes Groth16 proof size; PLONK or STARK instantiations would increase the proof component. Neither column accounts for chain-specific encoding overhead (e.g., RLP, SSZ, or calldata padding).

Component	PQC Sig Model	ZK-ACE
Signature / Proof	2,420–4,627 B	128–256 B (Groth16)
Public key (amortized)	1,312–2,592 B	0 B (commitment reused)
Identity commitment	—	32 B
Public inputs	—	$\approx$ 160 B
Total per TX	3,732–7,219 B	$\approx$320–448 B

- **Zero-knowledge proof:** A succinct proof attesting to authorization correctness. Modern ZK proof systems produce constant-size proofs, typically on the order of 128–256 bytes (Groth16) to approximately 1 KB (Plonk/STARKs), independent of the complexity of the underlying authorization semantics.
- **Public inputs:** A small set of consensus-visible values, including TxHash, domain, target, and rp_com. These inputs typically contribute on the order of 128–192 bytes.

Crucially, none of these artifacts scale with the size or internal structure of post-quantum signature schemes.

10.4 Order-of-Magnitude Comparison

Comparing the structural requirements of the two models yields a clear order-of-magnitude difference in on-chain authorization data, summarized in Table 3:

Based on standardized post-quantum signature sizes (ML-DSA level 2–5, Table 2) and representative ZK proof sizes (Groth16), the structural replacement implies a *per-transaction on-chain data reduction on the order of 10–20×* for the authorization artifacts. These figures follow directly from the byte-count comparison in Table 3 and do not depend on implementation-specific optimizations. Importantly, this reduction does not arise from compressing post-quantum signatures. Rather, it is achieved by *eliminating the need to place post-quantum signature material on-chain* and replacing it with succinct proofs of authorization semantics.

Comparison caveats. Table 3 presents a structural, protocol-level comparison intended to illustrate the order-of-magnitude difference in authorization artifact sizes. Several factors affect the precise ratio in any concrete deployment: (i) public-key amortization depends on whether the sender’s key is already known to the chain (repeat senders amortize to zero in the PQC

model, narrowing the gap); (ii) ZK proof size varies by proof system (Groth16 yields ≈ 128 – 256 B, while PLONK or STARK proofs are larger); and (iii) chain-specific encoding formats (RLP, SSZ, ABI encoding) add overhead to both models. The comparison is therefore best understood as a structural accounting of the *cryptographic artifact sizes* mandated by each authorization model, rather than as a prediction of concrete gas costs or byte counts on any specific chain.

11 Scalability Extensions

11.1 Batch Authorization

Multiple independent authorization statements from distinct identities can be combined into a single aggregated proof using batch-verification or proof-aggregation techniques. This amortizes verifier work and on-chain artifact count: instead of n individual proofs (each ≈ 128 – 256 bytes), a single batched proof attests to n authorization statements, reducing per-transaction verification cost and data overhead.

Batch authorization preserves the individual security guarantees of each component statement: knowledge soundness of the aggregated proof implies knowledge soundness of each constituent statement. The aggregator (which collects individual proofs and produces the batch proof) is untrusted and does not learn any private witness material.

11.2 Recursive Proof Composition

Recursive composition allows a top-level proof to attest to the validity of many base-level proofs [11, 12]. Applied to ZK-ACE, a recursive proof can verify k individual authorization proofs, producing a single constant-size proof regardless of k . This is particularly useful in rollup architectures, where a single recursive proof can authorize an entire batch of transactions to be posted to the base layer.

The recursion preserves all ZK-ACE security properties: if the recursive proof system is knowledge-sound, then the existence of an accepting recursive proof implies the existence of valid witnesses for all constituent authorizations. Cross-domain separation is maintained because domain tags propagate through the recursion.

12 Reference Implementation and Benchmarks

To validate the structural analysis of Sections 7.3 and 10, we provide a reference implementation of the ZK-ACE circuit and report single-threaded CPU benchmarks. The implementation is open-source and uses the arkworks library ecosystem [26] for BN254 arithmetic and Groth16 proving.

12.1 Implementation

The reference circuit implements all five constraints (C1)–(C5) using the arkworks `PoseidonSpongeVar` gadget with width $t = 3$, S-box exponent $\alpha = 17$, 8 full rounds, and 57 partial rounds over the BN254 scalar field. Groth16 [8] is used as the proof system. The circuit produces identical constraint counts (4,024 R1CS) for both the nonce-registry and nullifier-set replay modes, confirming that the replay-mode selection does not affect circuit complexity.

The implementation additionally includes a native (off-chain) Poseidon hash using identical parameters, ensuring functional equivalence between the in-circuit computation and the off-chain witness generation.

Table 4: Groth16 proof-system benchmarks for ZK-ACE (single-threaded, Apple M3 Pro).

Operation	Median	95% CI
Trusted setup (one-time)	120 ms	[111, 138] ms
Prove (per transaction)	63 ms	[61, 65] ms
Verify (per transaction)	651 μ s	[618, 703] μ s

Table 5: ACE runtime pipeline benchmarks (single-threaded, Apple M3 Pro). Phase 2 uses a mock prover; real Groth16 timings are in Table 4.

Operation	Batch size	Median time
HKDF key derivation (single)	1	1.16 μ s
Identity commitment (ID_{com})	1	270 ns
Attestation generate	1	2.25 μ s
Attestation verify	1	0.82 μ s
Phase 1a: AttestCheck	2,000	247 μ s (124 ns/tx)
Phase 1b: Execute	2,000	134 μ s (67 ns/tx)
State root (Merkle)	2,000	537 μ s
Block build	2,000	1.97 ms
Phase 2: mock prove [†]	2,000	4.16 ms
End-to-end pipeline [†]	2,000	7.56 ms
REV Argon2id seal/unseal	1	3.5 ms

[†]Mock prover: orchestration overhead only, not real ZK proof generation.

12.2 Benchmarks

All measurements were collected using Criterion.rs on an Apple M3 Pro (12-core, 36 GB RAM) running macOS 15.7, Rust 1.92, with single-threaded execution and default compiler optimizations (`--release`).

ZK proof operations. Table 4 reports the core Groth16 operations for a single authorization proof. All timings are Criterion.rs medians from `cargo bench -p zk-ace --bench prove_verify_bench`.

The verification time of ≈ 0.65 ms per proof is well within the budget for on-chain verification in both L1 and rollup settings. The proving time of ≈ 63 ms is for single-threaded CPU execution; prior work on GPU-accelerated Groth16 provers (e.g., Icicle, Bellman-GPU) reports 5–15 $\times$ speedups on comparable circuit sizes, which would project single-proof latency to ≈ 4 –13 ms, though this has not been measured for this specific circuit.

Attestation and pipeline operations. Table 5 reports timings for the identity-layer primitives and the consensus-critical pipeline stages, measured on the ACE runtime using `cargo bench -p ace-runtime --features test-utils --bench pipeline_bench`. **Note:** Phase 2 (proof generation) uses a deterministic mock prover that simulates per-transaction SHA-256 hashing and tree aggregation; it does *not* invoke real Groth16 proving. The mock-prover timings reflect the overhead of the proving *orchestration*—witness marshalling, tree construction, and data serialization—exclusive of the ZK proof computation itself, which is benchmarked separately in Table 4.

At batch size 2,000, the consensus-critical Attest–Execute path (Phase 1a + 1b + Merkle root) completes in ≈ 0.92 ms, well under 1% of a 400 ms slot window. The attestation-check

phase processes each transaction in ≈ 124 ns, consistent with the sub-microsecond HMAC-SHA256 verification expected from the design. The end-to-end pipeline (with mock proving) takes ≈ 7.56 ms; replacing the mock prover with real Groth16 proving would add ≈ 63 ms $\times$ 2,000 of prover work, which is parallelizable across GPU cores and off the critical consensus path (Section 11.1).

13 Limitations and Future Work

- **DIDP dependency.** The security of ZK-ACE is conditional on the assumed security properties of the underlying DIDP (Section 3.3). These properties—determinism, context isolation, and identity-root recovery hardness—follow from standard assumptions (collision resistance and high min-entropy of REV) when the derivation function is modeled as a random oracle. ACE-GF [1] is one concrete instantiation; independent analysis of any candidate DIDP would further strengthen the foundation of this work.
- **Nullifier-set state growth.** In the nullifier model, the global nullifier set grows monotonically with the number of authorization events. Long-term state management strategies (e.g., epoch-based pruning, Merkle-tree compaction) are an open engineering problem.
- **Single-platform benchmarks.** The benchmarks in Section 12 are collected on a single platform (Apple M3 Pro, single-threaded CPU). Gas-precise, chain-specific cost estimates and GPU-accelerated proving times remain to be measured. Cross-platform validation (e.g., x86, ARM server, WASM) and formal circuit audits are deferred to follow-up work.
- **Prover cost.** While verifier cost is low (≈ 0.65 ms), prover cost (≈ 63 ms single-threaded CPU) may be non-trivial on resource-constrained devices. Prior work on GPU-accelerated Groth16 provers reports 5–15 $\times$ speedups on comparable circuit sizes, but this has not been measured for the ZK-ACE circuit.
- **Identity revocation and rotation.** ZK-ACE as specified does not include an on-chain mechanism for identity revocation or commitment rotation. Extending the framework with revocation semantics compatible with the identity-commitment model is a direction for future work.
- **Privacy-throughput trade-off.** The choice between nonce and nullifier models involves a fundamental trade-off between ordering transparency and privacy. Hybrid models combining both strategies for different authorization classes merit further investigation.

14 Conclusion

ZK-ACE reframes post-quantum-ready blockchain authorization as an identity-bound proof problem rather than a signature-object verification problem. By replacing transaction-carried post-quantum signatures with zero-knowledge proofs of deterministic authorization, the resulting model preserves verifier-side semantic guarantees while eliminating consensus dependence on kilobyte-scale signature artifacts.

The construction achieves this through a minimal set of components: an identity commitment anchored on-chain, a circuit that enforces commitment consistency, target binding, transaction authorization, replay prevention, and domain separation, and a verification interface compatible with account-abstraction and rollup architectures. The formal security analysis demonstrates that authorization soundness, replay resistance, substitution resistance, and cross-domain separation all reduce to standard cryptographic assumptions.

The structural data accounting shows an order-of-magnitude reduction in per-transaction authorization data relative to direct post-quantum signature deployment—achieved not by compressing signatures, but by eliminating the need for signature objects in the consensus path entirely. A reference implementation confirms a total circuit size of 4,024 R1CS constraints

with sub-millisecond verification and sub-100 ms single-threaded proving on consumer hardware. With support for batch aggregation and recursive composition, ZK-ACE provides a scalable authorization architecture for post-quantum blockchain systems.

References

- [1] J. S. Wang. ACE-GF: A Generative Framework for Atomic Cryptographic Entities. arXiv preprint arXiv:2511.20505, 2025. <https://arxiv.org/abs/2511.20505>.
- [2] NIST. FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA). Federal Information Processing Standards, 2024.
- [3] NIST. FIPS 205: Stateless Hash-Based Digital Signature Standard (SLH-DSA). Federal Information Processing Standards, 2024.
- [4] NIST. Draft FIPS 206: FFT over NTRU-Lattice-Based Digital Signature Standard (FN-DSA). Federal Information Processing Standards (draft), 2024.
- [5] A. Biryukov, D. Dinu, and D. Khovratovich. Argon2: The memory-hard function for password hashing and other applications. In *IEEE EuroS&P Workshops*, 2016.
- [6] L. Grassi, D. Kales, D. Khovratovich, A. Roy, C. Rechberger, and M. Schofnegger. Poseidon: A new hash function for zero-knowledge proof systems. In *USENIX Security Symposium*, 2021.
- [7] A. Gabizon, Z. J. Williamson, and O. Ciobotaru. PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge. *IACR ePrint* 2019/953, 2019.
- [8] J. Groth. On the Size of Pairing-based Non-interactive Arguments. In *EUROCRYPT*, 2016.
- [9] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev. Scalable, transparent, and post-quantum secure computational integrity. *IACR ePrint* 2018/046, 2018.
- [10] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *IEEE S&P*, 2018.
- [11] S. Bowe, J. Grigg, and D. Hopwood. Recursive proof composition without a trusted setup. *IACR ePrint* 2019/1021, 2019.
- [12] A. Kothapalli, S. Setty, and I. Tzialla. Nova: Recursive zero-knowledge arguments from folding schemes. In *CRYPTO*, 2022.
- [13] E. B. Sasson, A. Chiesa, C. Garman, M. Green, I. Miers, E. Tromer, and M. Virza. Zero-cash: Decentralized anonymous payments from Bitcoin. In *IEEE S&P*, 2014.
- [14] A. Pertsev, R. Semenov, and R. Storm. Tornado Cash privacy solution. White Paper, 2019.
- [15] Privacy and Scaling Explorations (PSE), Ethereum Foundation. Semaphore: Zero-knowledge signaling on Ethereum. <https://semaphore.pse.dev/>, 2020–2024.
- [16] V. Buterin, Y. Weiss, D. Kristjansson, N. Luca, T. Silberstein, and M. Tirosh. ERC-4337: Account abstraction using alt mempool. Ethereum Improvement Proposal, 2021.

- [17] V. Buterin, D. Feist, D. Loerakker, G. Kadianakis, M. Garnett, M. Taiwo Adeniyi, and A. Schoedon. EIP-4844: Shard blob transactions. Ethereum Improvement Proposal, 2022.
- [18] 0xPARC. circom-ecdsa: Proving ECDSA with zkSNARKs. <https://github.com/0xPARC/circom-ecdsa>, 2022.
- [19] D. Boneh and M. Franklin. Identity-based encryption from the Weil pairing. In *CRYPTO*, 2001.
- [20] A. Shamir. Identity-based cryptosystems and signature schemes. In *CRYPTO*, 1984.
- [21] H. Krawczyk and P. Eronen. HMAC-based Extract-and-Expand Key Derivation Function (HKDF). RFC 5869, 2010.
- [22] S. Goldwasser, S. Micali, and C. Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1):186–208, 1989.
- [23] M. Bellare and P. Rogaway. The security of triple encryption and a framework for code-based game-playing proofs. In *EUROCRYPT*, 2006.
- [24] D. Boneh and V. Shoup. *A Graduate Course in Applied Cryptography*. Version 0.6, 2023. <https://toc.cryptobook.us/>.
- [25] A. Chiesa, G. Fenzi, and J. Weissenberg. Zero-Knowledge IOPPs for Constrained Interleaved Codes. *IACR ePrint* 2026/391, 2026.
- [26] arkworks contributors. arkworks: An ecosystem for developing and programming with zkSNARKs. <https://arkworks.rs/>, 2021–2024.